

KernelGuard Phase 3 Project Report

OS (TCS 502) and DBMS (TCS 503) | Academic session 2026–27

Graphic Era (Deemed to be University), Dehradun

Project field	Submission detail
Team ID	OSDBMS-V-2026-T298
Team name	KernelGuard
Mentor	(Prof.)Dr.Mohammad Wazid
Source snapshot	Phase 3 completed project

Team member	Student ID	Area presented
Arshpreet Singh	24021835	Linux audit and host collection
Mohd Ahmed Khan	24021365	Database and persistence
Ankit	240212134	Detection rules and evaluation
Mohd Shoaib	240211124	Dashboard and review

Project Abstract

KernelGuard Phase 3 extends the two-rule audit prototype into a local Linux host activity monitor. It combines normalized audit events with periodic user, session, process and USB observations. Five configured policies create alerts that retain their supporting events and policy snapshots. An authenticated administrator can review alerts and record USB approval decisions without erasing earlier history. The application uses relational transactions and supports both a SQLite demonstration and MySQL-backed Linux deployments.

Updated Project Approach and Architecture

Linux Audit supplies file, authentication and privileged-command metadata. Inventory polling reads account, session, process and USB information from operating-system facilities. A writer lock serializes cooperating writes. Transactions keep audit events with their checkpoints, and inventory transitions with their alerts and device records. Process identity includes host, source, boot ID, PID and start time so PID reuse does not merge unrelated processes. The web layer exposes evidence, inventory and administrator review. MySQL foreign keys preserve relationships, while Flask sessions and CSRF checks protect state-changing routes [1–4].

Component	Phase 3 implementation
Collection	Audit parser plus pwd, procfs, psutil and pyudev inventory
Detection	Failed login, after-hours file access, bulk distinct paths, selected privileged commands, unapproved USB
Persistence	17 relational MySQL tables, evidence links, policy snapshots and review history
Interface	Authenticated dashboard, alert review and device approval

Tasks Completed

The source includes audit and inventory collectors, five rule families, replay handling, strict configuration, relational schema, SQL analysis queries, administrator authentication, CSRF protection, alert acknowledgement and reopening, USB enrollment and revocation, automated tests, evaluation scripts and Linux setup instructions. Responsibility areas below are an allocation for presentation and maintenance, not proof of individual authorship.

Member	Presentation responsibility
Arshpreet Singh	Linux audit and host collection
Mohd Ahmed Khan	Database and persistence
Ankit	Detection rules and evaluation
Mohd Shoaib	Dashboard and review

Testing and Validation Status

The saved Windows evaluation matched 11 of 11 hand-authored expected policy outcomes. Its disk-backed SQLite workload processed 500 synthetic events and created 100 alerts in 0.4806 seconds, or about 1,040 events per second, excluding source collection, inventory and web rendering. This is a local synthetic measurement, not production throughput or field accuracy. On Kali, the project records live protected-file, privileged-command, bulk-read and failed-login checks, restart replay, inventory ingestion and administrator review. A separate Oracle MySQL 8.4.11 deployment and disposable integration test passed on 2 October 2026.

Evidence	Recorded result	Interpretation
Policy cases	11/11 expected outcomes	Hand-authored synthetic cases
Ingestion workload	500 events, 100 alerts, 0.4806 s	SQLite, one transaction, excludes collection
Live Kali checks	Audit, inventory and review flows observed	Controlled lab actions
Oracle MySQL 8.4	Separate deployment and integration test passed	On the recorded Kali host

Challenges and Roadblocks

Audit output needs record grouping, boot-aware identity and replay suppression. Process scans can be incomplete because a process exits or permissions deny inspection; an incomplete scan retains a warning instead of fabricating a disappearance. USB sampling can miss a short attach/remove cycle, and device identifiers may be reused or cloned. Legitimate backup reads or an authorized selected command can meet a policy, so a human must review linked evidence before interpreting an alert.

Project Outcomes and Progress

The software, demo and documented Linux deployments are implemented. The result is a reviewable host-monitoring prototype with retained evidence and decision history. Physical USB removal/reconnection and enrollment with a spare device remain unverified on the available Kali host. Live USB enumeration failure handling also remains to be exercised. The source does not support a claim of real-world precision, recall, attack prevention or a completed hardware acceptance test.

Codebase Information and Deliverables

Source folder: C:\KernalGuard. Core modules are kernelguard/core.py, activity_rules.py, parser.py, inventory/, auth.py, reviews.py and web.py. Reproducible artifacts include fixtures/, sql/, linux/, tests/, scripts/evaluate.py and docs/ACCEPTANCE.md. The local Git branch is main. The two visible commits are a project snapshot and MySQL deployment documentation; current working-tree edits are not submission history. A public repository URL and individual commit counts were not available and should be added from the team's verified Git record.

Reproduction and Interpretation

For an offline demonstration, initialize a fresh local database, run the project demo command and start the localhost dashboard. The fixture gives 21 events, five alerts, two users, one audit session, three processes and one USB device. These values are explicitly synthetic and should be shown with their demo label. Run `python -m scripts.evaluate` separately to regenerate policy cases and a local ingestion timing. A new run may produce a different timing, so the saved 0.4806-second result should be described as the recorded run.

For live acceptance, use the Linux setup and acceptance documents with a disposable lab environment. Check the configured audit rule, collector status and database connection before performing controlled actions. Retain the raw audit identity and stored event together, and trace the event into the alert evidence page. For inventory, record whether the snapshot was complete and whether a device was approved at the time of observation. A missing process in an incomplete scan should not be interpreted as an exit.

The evaluation cases test whether the configured rules produce the expected outcome for selected synthetic scenarios. They do not estimate how often ordinary users would produce an alert in deployment. A field accuracy study would need independently labelled real activity, a fixed policy configuration and a larger observation period. The two documented legitimate policy matches illustrate why the administrator review history matters.

Originality and Attribution

The report uses fresh, project-specific wording and references the technologies used. The repository's originality record states that AI assisted initial implementation and review, and that no third-party project repository was imported in the visible session. That record is not a similarity certificate. No institutional plagiarism scan has been run for these deliverables. The team should retain accurate attribution and explain its own contribution during the evaluation.

References

- [1] Red Hat, Understanding Audit Log Files, https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/security_guide/sec-understanding_audit_log_files
- [2] SQLAlchemy 2.0, Working with Engines and Connections, <https://docs.sqlalchemy.org/en/20/core/connections.html>
- [3] MySQL 8.4 Reference Manual, FOREIGN KEY Constraints, <https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html>
- [4] Flask Documentation, Security Considerations, <https://flask.palletsprojects.com/en/stable/web-security/>
- [5] KernelGuard source, docs/ARCHITECTURE.md, docs/EVALUATION.md, docs/LIVE_AUDIT_CHECK.md and docs/MYSQL84_DOCKER.md in C:\KernalGuard.